PROJECT DOCUMENTATION

Dynamic Trader Architecture Guide

Complete map of the hybrid stack for perpetual-futures research on OKX BTC-USDT-SWAP. Rust is the Nautilus-matching engine. Go (`dt`) is the product layer for ranking, walk-forward policy, browse UI, and reports. Written for fullstack developers who did not author the Rust.

MARKET	OKX BTC-USDT-SWAP (linear, USDT-settled)
ENGINE	Rust 2024 edition · NautilusTrader 0.65.0 behind feature `verify`
PRODUCT CLI	Go module `github.com/ramtin/dynamic-trader` · binary `dt`
BRANCH	`ramtin/cleanup-docs-b567`
HOW TO READ	This PDF, or open `docs/architecture.html` in a browser

Contents

1. Mental model and system pipeline
2. Repository layout
3. End-to-end data flow
4. Rust library modules (every file)
5. Rust binaries
6. On-disk formats
7. simulate JSONL protocol
8. Go product packages
9. Feature verify / Nautilus boundary
10. Config and safety
11. Standard research workflow
12. Cleanup removals
13. Where to edit what

1. Mental model and system pipeline

OKX zips → Rust tape → scan .bin → Go walk-forward → simulate → JSON / HTML

Rust = engine

Download archives, build the tick tape, scan millions of RSI × TP/SL combos, batch-simulate fills, optionally verify with NautilusTrader.

Go = product (`dt`)

Rank windows, run N/M walk-forward policy, serve browse HTTP, write reports. Changes compile in seconds without linking Nautilus.

Rule of thumb. Change fill rules in Rust (`src/scan.rs` + `src/strategy.rs`) and confirm with `verify` . Change how winners are picked in Go (`internal/walkforward` , `internal/score`).

2. Repository layout

```
ppo-trader/
├── Cargo.toml                 Rust crate + optional feature verify
├── go.mod / cmd/dt/           Go product CLI
├── config/testnet.toml        Fees, paper_only, max leverage
├── scripts/
│   ├── build_dt.sh            macOS-safe Go build (CGO + external linker)
│   ├── build_architecture_pdf.sh
│   └── price_iter.sh          Fast price-path iteration
├── docs/
│   ├── architecture.html      Source of this guide
│   ├── ARCHITECTURE.pdf       Generated PDF
│   └── DT.md                  Short CLI cheat sheet
├── src/                       Rust library (shared by all engine bins)
├── src/bin/                   Rust binaries
└── internal/                  Go packages used by dt

data/   gitignored — OKX archives + scan outputs live only on disk
```

3. End-to-end data flow

Research path (≈99% of compute)

1. `download` fetches OKX daily trade zips into `data/okx/BTC-USDT-SWAP/archives/` and verifies trade-ID continuity + volume vs OKX's official 1D candle.
2. `Tape::load` converts every trade into Nautilus L1 matching prices and builds a segment tree for TP/SL `first_hit` .
3. `scan --bucket window` enumerates RSI pairs × timeframe × TP/SL × leverage into fixed-size `window_*.bin` files plus `meta.json` .
4. Go `dt walk-forward` ranks those bins, picks finalists, batch-calls Rust `simulate` (JSONL) for closed trades, runs N/M score policy, writes JSON/HTML.

Truth path (slow)

Only the last / best combos go through Nautilus via `verify` or `backtest` . Same fill rules as the scanner; disagreement fails the run. The price path is pure Go and does not use Nautilus.

4. Rust library modules (`src/`)

Every library file, approximate size, and responsibility.

FILE	LINES	RESPONSIBILITY
`okx_history.rs`	551	OKX download/verify, Trade/Candle, known gaps & voided IDs, archive paths
`tape.rs`	383	Tick matching tape + first_hit tree (Nautilus L1 rules, no Nautilus link)
`scan.rs`	523	Fast RSI-cross + exits simulator; Outcome / ClosedTrade / Record
`tpsl_grid.rs`	772	Vectorized single-TP×SL×lev grid (Entries / Frame / Grid)
`window_score.rs`	849	WindowStats (20-byte records), ScoreModel budget-10 fit
`exits.rs`	249	Stop / TP ladder math + ExitProfile scale-outs
`filters.rs`	226	EMA trend + ATR filters wrapping crosses
`grid.rs`	185	CLI parsers for lengths/ladders + RSI series helpers
`rsi.rs`	90	Wilder RSI
`rsi_cross.rs`	114	Fast/slow RSI cross signals
`pine_rsi.rs`	146	TradingView “RSI Strategy” port
`fills.rs`	26	Fill / FillKind — no Nautilus dependency
`signals.rs`	20	Signal / BarSignals — no Nautilus dependency
`config.rs`	91	TOML load; rejects live trading & bad leverage
`verify_cmp.rs`	60	Compare scanner vs Nautilus fills
`tradingview.rs`	371	TV WebSocket/HTTP client for tv_check
`strategy.rs`	319	Nautilus ReversalStrategy (feature verify)
`backtest.rs`	259	Nautilus BacktestEngine wrapper (feature verify)
`lib.rs`	21	Module exports; gates backtest/strategy on verify

Types you will see everywhere

- `okx_history::Trade` / `Candle` — raw market data
- `tape::Tape` — matching prices + `first_hit(from, to, level, up)`
- `exits::Exits` — `{ stop_pct, targets: [(pct, share)], breakeven }`
- `scan::simulate` / `simulate_logged` — fast replay API
- `fills::Fill` — one fill (time, kind, side, size, price, fee)
- `window_score::WindowStats` — per-combo per-window score inputs
- `signals::BarSignals` — trait implemented by RSI / Pine / FilteredCross

5. Rust binaries (`src/bin/`)

BINARY	LINES	NAUTILUS?	WHAT IT DOES
`download`	52	No	Fetch + verify OKX day archives. Args: `START END` (end exclusive), `--dir`
`scan`	~2175	No	`--bucket window|month` . Writes meta.json + .bin records. Split: main.rs / window.rs / month.rs
`simulate`	438	No	Load tape once; JSONL stdin → JSONL stdout batch replay
`verify`	192	Yes	One combo through Nautilus; compare fills JSON
`backtest`	207	Yes	Pine RSI strategy table via Nautilus

BINARY	LINES	NAUTILUS?	WHAT IT DOES
`tv_check`	694	No engine	Compare candles/RSI/trades vs TradingView (needs .env cookies)

`window_nm_walk_forward` and `price_model.rs` were removed after the Go port (`dt walk-forward` / `dt price-walk`).

6. On-disk formats

OKX archives

```
data/okx/BTC-USDT-SWAP/archives/
  BTC-USDT-SWAP-trades-2026-09-01.zip   # or gz
  (+ verified marker used by is_verified())
```

An archive day runs 16:00 UTC previous day → 16:00 UTC (same window as OKX's official 1D candle).

Window scan (`--bucket window`)

- `meta.json` — start/end, timeframes, lengths, TP/SL/leverage grids, window list, fees, size_btc, scan_version (1 = dense, 3 = staged)
- `window_*.bin` — one **20-byte little-endian** record per combo:

```
profit f32 | drop f32 | worst f32 | hold_hours f32 | trades u16 | wins u16
(window_score::RECORD = 20)
```

Month scan (`--bucket month`)

- `month_YYYY-MM.bin` , `all.bin` — **12-byte LE** records: `sum f32 | drop f32 | trades u32`

7. `simulate` JSONL protocol

> **Critical.** Walk-forward must batch. Never spawn one process per combo.

Request line

```
{"id":"42","minutes":60,"fast":14,"slow":50,
 "exits":{"stop_pct":1.0,"targets":[[1.0,1.0]],"breakeven":false},
 "leverage":2,"logged":true,"record_fills":false}
```

Response line

Includes `outcome` (realized, fees, total, …), optional `closed[]` trades (`exit_ms` , `entry_ms` , `net`), optional `fills[]` .

CLI flags: `--start --end --config --data --size-btc --balance-usdt --jobs` . Send `{"flush":true}` to force a batch flush.

8. Go product packages

PACKAGE	LINES	JOB
`cmd/dt`	~210	CLI router (logrus via logx)

PACKAGE	LINES	JOB
`internal/engine`	~280	Find/exec Rust bins; Simulate JSONL client
`internal/browse`	1166	HTTP browser over window/month bins
`internal/walkforward`	~985	N/M score-path walk-forward
`internal/pricewalk`	~1200	Candle OLS + oracle exits (former Rust --path price)
`internal/pricemodel`	~220	OLS candle-return features
`internal/candles`	~330	OHLC aggregate, M1C1 cache, archive trades
`internal/score`	459	Budget-10 ScoreModel (Rust parity tests)
`internal/report`	~80	Embed score + price HTML; regenerate from JSON
`internal/logx`	~30	Shared logrus logger
`internal/config`	~50	TOML paper config (go-toml)
`internal/exits`	~115	TP/SL ladder types shared by price path

Engine discovery order

```
$DT_ENGINE_DIR
./target/relfast
./target/release
./target/debug
$PATH
```

Commands

```
./scripts/build_dt.sh
./dt download|scan|simulate|verify|backtest|tv-check …
./dt browse --scan DIR [--port N] [--layout window|month|auto]
./dt walk-forward --scan DIR [score-path flags…]
./dt price-walk --scan DIR [price flags…]   # pure Go; see scripts/price_iter.sh
./dt report --json PATH
```

9. Feature `verify` and the Nautilus boundary

> Without `--features verify` (or with `--no-default-features`), `backtest` and `strategy` are not compiled and nautilus crates are not linked. Scanner rebuilds drop from painful LTO toward ~5–15s with the `relfast` profile.

```
# Fast engine only
cargo build --profile relfast --no-default-features \
  --bin download --bin scan --bin simulate

# Need Nautilus
cargo build --profile relfast --features verify \
  --bin verify --bin backtest
```

10. Config and safety

`config/testnet.toml`

- `risk.paper_only` must be `true` (live rejected by validation)

- `risk.max_leverage` validated (1–10 in config)
- `venues.okx.maker_fee_bps` / `taker_fee_bps`

Secrets only in `.env` (gitignored). `tv_check` needs TradingView `sessionid` cookies. Never commit keys or seed phrases.

11. Standard research workflow

```
# 1) Two years of trades (end exclusive)
cargo run --release --bin download -- 2024-10-31 2026-10-31

# 2) Window scan
cargo run --release --bin scan -- --bucket window \
  --start 2025-10-01 --end 2026-10-01 --hours 24

# 3) Browse
./scripts/build_dt.sh
./dt browse --scan data/windowscan/<dir> --port 8091

# 4) Score walk-forward (Go)
./dt walk-forward --scan data/windowscan/<dir>

# 5) Price path (Go; M1C1 candle cache under scan/.cache/)
./dt price-walk --scan data/windowscan/<dir> --price-bars 15m --price-strategy 1 --verify 0
# or: ./scripts/price_iter.sh
```

12. Cleanup: removed vs replaced

Deleted as unused after the hybrid split (~7.8k lines):

- `browse_full_scan` / `browse_window_scan` → replaced by `dt browse`
- Thin `full_scan` / `window_scan` wrappers → use `scan --bucket …`
- Legacy orchestration: `rsi_combos`, `walk_forward`, `walk_forward_trades`, `full_walk_forward`, `window_walk_forward` (+ HTML reports)

Also removed in the price-path Go port: `window_nm_walk_forward`, `price_model.rs`, and the embedded HTML/JS under `src/bin/`. Still present: engine bins only (`download`, `scan`, `simulate`, `verify`, `backtest`, `tv_check`).

13. Where to edit what

YOU WANT TO…	EDIT
Change how TP/SL fills	`src/scan.rs + src/strategy.rs, then verify`
Change OKX download / gaps	`src/okx_history.rs`
Change scan grid / staged survivors	`src/bin/scan/window.rs`
Change ranking formula	`src/window_score.rs and internal/score`
Change N/M walk-forward policy	`internal/walkforward/nm.go`
Change browse UI	`internal/browse/`
Change price-path experiment	`internal/pricewalk/ + internal/pricemodel/`

Regenerate this PDF: `./scripts/build_architecture_pdf.sh`
On-screen reading: open `docs/architecture.html` in a browser.
CLI cheat sheet: `docs/DT.md`